CHECKOUT GUARDIAN

Technical Requirements Document (TRD) — Version 1.0

Project: Checkout Guardian

Context: Implementation-Ready Technical Specification

Status: Approved / Architecture Ready

Date: October 2026

Table of Contents

1. Technical Overview
2. System Architecture
3. Technology Stack and Justification
4. Detailed Component Specifications
5. Data Flow and Processing Pipeline
6. API Requirements and Specifications
7. PayPal Integration Architecture
8. Database Design
9. AI and Risk Engine Technical Design
10. Security Architecture
11. Error Handling and Reliability
12. Non-Functional Technical Requirements
13. Frontend and Browser Extension Requirements
14. Project Folder Structure
15. Development and Deployment Requirements
16. Testing Strategy
17. Performance and Risk Evaluation
18. Technical Limitations and Future Improvements
19. Four-Member Technical Work Allocation
20. Technical Acceptance Criteria

1. Technical Overview

Project Introduction: Checkout Guardian is an intelligent pre-payment security layer designed for the PayPal AI Hackathon 2026. It protects consumers from sophisticated e-commerce fraud by evaluating website authenticity, merchant credentials, product pricing anomalies, and payment recipient consistency before financial settlement occurs.

Technical Objectives: Deliver an end-to-end verifiable fraud-detection and payment-intervention pipeline; implement the core *Merchant Identity & Offer Consistency Engine*; integrate seamlessly with PayPal JavaScript SDK and Orders API in Sandbox; maintain sub-2.0s evaluation latency.

Architectural Principles: Strict separation of concerns between AI semantic extraction and deterministic payment decisions; fail-open safety with warnings during transient API timeouts; strict data minimization (zero raw PII or card storage).

2. System Architecture

The system operates across a client-facing tier (React SPA / Chrome Extension), an orchestration tier (Express.js API Router), a core logic tier (Authenticity, Merchant Verification, Consistency, and Risk Engines), and a storage/external tier (PostgreSQL, LLM API, PayPal Sandbox).

3. Technology Stack and Justification

Technology	Purpose	Why Appropriate
React.js & Vite	Frontend SPA & Dashboard	Fast bundling, component-based UI development.
Node.js & Express.js	Backend API Server	Asynchronous I/O, rapid API prototyping, JS uniformity.
PostgreSQL	Relational Database	ACID compliance, robust relational integrity for audit logs.
OpenAI / LLM API	AI Analysis Service	Contextual evaluation of deceptive promotional text and risk rationales.
PayPal JS SDK & Orders API	Payment Rails & Capture Control	Direct sandbox integration for secure checkout capture management.

4. Detailed Component Specifications

A. Website Authenticity Module

Extracts target URL, inspects SSL certificate metadata, queries WHOIS creation dates, and computes string similarity against known retail brands.

B. Merchant Identity Verification Module

Extracts footer business names, contact info, and legal disclaimers to cross-reference against trusted entity registries.

C. Product and Price Intelligence Module

Compares advertised prices against baseline market references to flag predatory outlier discounts.

D. Merchant Identity & Offer Consistency Engine (Central Innovation)

Compares claimed storefront branding against the actual `PAYEE_MERCHANT_ID` returned by PayPal order creation. Mismatches trigger immediate high-risk weighting.

E. AI Analysis Module & Risk Assessment Engine

Synthesizes heuristic scores and LLM semantic outputs into a weighted 0–100 risk score, enforcing JSON schema validation and separating AI extractions from deterministic capture control.

5. Data Flow and Processing Pipeline

The lifecycle transitions from metadata ingestion during page browse, through preliminary heuristic scoring, to order initiation via PayPal, and finally backend consistency verification before authorizing or pausing capture execution.

6. API Requirements and Specifications

Core backend endpoints include:

- `POST /api/v1/websites/analyze` — Ingests website metadata and domain metrics.
- `POST /api/v1/merchants/verify` — Cross-references merchant registration credentials.
- `POST /api/v1/risk/assess` — Executes full consistency check and computes risk score.
- `POST /api/v1/paypal/orders/capture` — Validates risk decision before authorizing PayPal capture.

```
{
  "success": false,
  "error": "PAYMENT_CAPTURE_PAUSED",
```

```
  "message": "Checkout Guardian intervened due to high merchant identity mismatch risk."
}
```

7. PayPal Integration Architecture

Utilizes the PayPal JavaScript SDK and Orders API in Sandbox. The backend API **must never** execute the capture endpoint if the Risk Assessment Engine returns a high-risk mismatch verdict, ensuring automated financial protection.

8. Database Design

Relational schema managed via PostgreSQL includes tables for `users`, `websites`, `merchants`, `assessments`, `risk_indicators`, `transactions`, and `audit_logs` with indexed foreign keys and strict data minimization constraints.

9. AI and Risk Engine Technical Design

Employs strict JSON schema validation for LLM outputs. Decision thresholds: 0–39 (Low Risk: Allow), 40–74 (Medium Risk: Warn), 75–100 (High Risk: Pause Capture).

10. Security Architecture

Client secrets and API keys secured in backend environment variables. Input sanitization prevents XSS and SSRF. CORS policies restrict unauthorized client origins.

11. Error Handling and Reliability

Transient WHOIS or LLM timeouts trigger fail-open fallback behavior with neutral scoring rather than blocking legitimate user transactions.

12. Non-Functional Technical Requirements

Target API response time < 2.0 seconds; 99.9% availability target during hackathon presentation; TLS 1.3 transport encryption enforced.

13. Frontend and Browser Extension Requirements

React SPA dashboard for history and audit logs; Manifest V3 Chrome Extension requesting `activeTab` and `scripting` permissions for safe DOM metadata extraction.

14. Project Folder Structure

```
checkout-guardian/
├── backend/ (Express app, services, models)
├── frontend/ (React SPA & dashboard)
├── extension/ (Manifest V3 Chrome extension)
└── shared/ (Common types and schemas)
```

15. Development and Deployment Requirements

Local setup via npm workspace installation, PostgreSQL configuration, and environment variable setup for PayPal Sandbox credentials and LLM API keys.

16. Testing Strategy

Jest unit tests for scoring logic, Supertest API integration tests, and live end-to-end scenario validation against PayPal Sandbox.

17. Performance and Risk Evaluation

Evaluated against benchmark genuine, suspicious, and impersonator test profiles, targeting 100% detection accuracy on fraudulent simulations.

18. Technical Limitations and Future Improvements

MVP focuses on controlled sandbox verification. Future commercial iterations will integrate global corporate registries and gateway-level web telemetry.

19. Four-Member Technical Work Allocation

Team Member	Core Focus Area	Assigned Modules
Member 1	AI & Intelligence Lead	Website Authenticity, LLM Prompts, Scraping
Member 2	Risk & Consistency Architect	Merchant Verification, Consistency Engine, Scoring
Member 3	Backend & Payment Lead	Express API, PostgreSQL Schema, PayPal Sandbox
Member 4	Frontend & Extension Lead	React SPA, Chrome Extension, UI Modals

20. Technical Acceptance Criteria

Successful end-to-end demonstration featuring automated website analysis, merchant identity consistency checking, risk scoring, and verified payment capture pausing within PayPal Sandbox.

Checkout Guardian — PayPal AI Hackathon 2026 | Technical Specification